

Adopting Agile in an FDA Regulated Environment

Rod Rasmussen Abbott Abbott Park, IL, USA Rodney.Rasmussen @abbott.com	Tim Hughes AgileTek LLC Des Plaines, IL, USA thughes@agiletek.com	J.R. Jenks AgileTek LLC Des Plaines, IL, USA jrjenks@agiletek.com	John Skach AgileTek LLC Des Plaines, IL, USA jskach@agiletek.com
--	--	--	---

Abstract

This paper is an experience report describing Abbott's adoption of agile software development practices in its molecular diagnostics division. We will compare two medical device projects; one before agile and one after. Both of these projects required submission to the FDA (the U.S. Food and Drug Administration). We will describe the adoption of agile practices from realization of the need to the selection of a mentor to implementation and fine-tuning and finally to results and lessons learned. This experience has convinced us that an agile approach is the approach best suited to development of FDA-regulated medical devices.

1. Introduction

Abbott's molecular diagnostics business provides physicians with critical information based on the early detection of pathogens and subtle changes in patients' genes and chromosomes, allowing for earlier diagnosis, selection of appropriate therapies and monitoring of disease progression. [1]

AgileTek is a custom software engineering firm. AgileTek focuses on clients that are developing life- or business-critical new products and services. AgileTek's methodology, Agile+, is an agile process that leverages the speed and cost-effectiveness of agile methods while at the same time incorporating features that make it a robust, highly-disciplined process suitable for large, mission critical projects and even for projects in highly regulated industries such as health care and aerospace. [2]

The projects discussed in this paper produce software for medical device products. They are classified as Class III devices, the most stringent FDA regulatory category for devices.

1.1. Looking For A Better Way

Early in 1994 Abbott Diagnostics Division (ADD) initiated the requirements definition process for a "next generation" product family. The family of medical devices would integrate disciplines of immunoassay and clinical chemistry into a single work cell, use common reagents and disposables, and address the low, medium, and high volume testing segments. In parallel to the requirements definition effort, software architects and analysts would complete an in-depth assessment and selection, of object oriented technologies and tools that would ultimately be used as the basis for implementation of the product family software. A pilot project using Shlaer-Mellor OOA notation, a commercially available CASE tool, and a custom homegrown OOA model translation engine completed mid-year 1995, providing proof of concept for a model based software engineering approach. January 22, 1999, the first of the product family systems, ARCHITECT i2000 was launched.

In January of 2004, Abbott's Molecular Diagnostics business began working in earnest on a nucleic acid purification platform and a companion real-time PCR analyzer, collectively known as the m2000. Prior to this point, in July of 2003, the core m2000 software team, all members with previous ARCHITECT i2000 experience, was evaluating options for software process, implementation technologies, and software architecture. The retrospective assessment of the ARCHITECT i2000 program yielded the following observations:

- At time of launch, ARCHITECT was staffed by nearly 100 FTEs working in some software function. Each was a specialist focused on either embedded development, UI and data management development, simulation development, integration testing, functional

testing, automated testing, on-line help, configuration management, or administration of infrastructure.

- The process required to build the ARCHITECT i2000 software was cumbersome and did not readily facilitate individual developers running private builds for debugging purposes. As a result, a higher incidence of defects likely escaped developer verification, and were subsequently identified and resolved during integration testing.
- Construction of builds was a two step process that typically ran overnight
 - Automated code generation translating OOA models to C++ source
 - Compilation and linking translated C++ source into executables
- Although ARCHITECT i2000 was developed incrementally, each increment was at least 6 months and often longer. The subsequent integration process for each incremental release was difficult and time consuming.
- Most of the tools used for ARCHITECT i2000 software were organically developed. Dedicated resources were required to maintain and / or evolve the tools. Customized tool training was required for new personnel. The commercial CASE tool used for specification of the Shlaer-Mellor OOA models was no longer supported by the vendor.
- Collaboration of Booch, Rumbaugh, and Jacobson at Rational resulted in emergence of UML, in essence obsoleting Shlaer-Mellor as a mainstream OO notation. Finding resources interested in working with Shlaer-Mellor notation to staff a new development project would be challenging.

2. Adopting Agile

By September 2003, the core m2000 software team had made several technology choices and initiated development of the product software architecture. Additionally, the team had completed a cursory review of various publications focused on software development process. Numerous books and journal articles described various “agile” approaches and techniques, all claiming to provide improved organizational productivity and higher quality software. Additionally, agile processes claimed to be able to readily accommodate emerging or changing requirements throughout the development lifecycle. This had a lot of appeal to the m2000 software team:

- The molecular segment of the diagnostics market was growing rapidly. Continual emergence of new delivery and detection technologies in addition to discovery of new diagnostic markers were part of the landscape. The ability to respond to new or changing requirements would be essential.
- Resource availability for m2000 software would be a fraction of that allocated to ARCHITECT i2000. Do more with less was a requirement in order to successfully commercialize the target product in a timely fashion.

The decision to adopt a modern software development tool set (.NET, C#, ADO.NET, Windows Forms, SQL, UML) and move to an agile approach had a lot of promise. However, the m2000 team had no practical experience with agile processes and putting concepts from books and journals into practice is often easier said than done. To address this gap, the team solicited help from AgileTek, a software engineering firm with extensive hands-on experience applying agile approaches to complex software development.

By the end of 2003, AgileTek had trained the core team on their approach to agile software development (Agile+) and conducted workshops on software architecture definition and high level application design. Additionally, AgileTek reviewed the m2000 software architecture and process definition as the team defined and evolved them.

3. Practicing Agile

In January of 2004, the m2000 software team put agile into practice. The process as initially defined by the team included the following practices:

- Fixed and short duration iterations (4 to 10 weeks) delivering functional software.
- Automated continuous build, unit test, and integration
- Daily team meeting providing high visibility of status and promoting concept of collective ownership
- Retrospective assessment at conclusion of each iteration by key stakeholders

The software development process as defined by the m2000 core team emphasized the concept of time-boxing. The overall project from inception to commercial launch was subdivided into increments.

Each increment resulted in delivery of executable software that was intended for distribution to one or more stakeholders outside of the software organization. Development of an increment consisted of completing one or more software iterations followed by a verification and validation phase. Each iteration delivered functional software. Additionally, an iteration generated key artifacts demonstrating compliance with design control processes imposed by various regulatory agencies. The verification and validation phase included formal execution of functional testing, formalized design reviews, approval of artifacts supporting the design control process, and update of the project's design history file.

The initial software increment was completed over five iterations concluding in August of 2004 with a release intended for internal use by the projects assay development organization. Follow on releases included a market evaluation and validation release in January of 2005, commercial launch of the m2000rt real-time PCR system in May of 2005, and commercial launch of the m2000sp sample preparation system in December of 2005. Post launch commercial releases were delivered on average, every 6-months, to further enhance the m2000 product.

4. Fine-Tuning Agile

The initial increment of software developed by the m2000 team consisted of five iterations. The duration of each iteration by design was different ranging from 4 to 10 weeks. The optimal duration as determined by the team was between 6 and 8 weeks. Given the inherent documentation requirements of medical device development, iterations shorter than 6 weeks did not generate sufficient velocity. Iterations much longer than 8 weeks provided opportunities for loss of organizational focus.

Other process modifications identified during retrospective stakeholder reviews and adopted over time included addition of weekly goals and a shift from scenario based planning to feature based planning.

Weekly goals were established for each week of a given iteration. These higher level objectives were reviewed at the end of each week during the daily meeting and provided additional focus on team objectives.

The feature based approach better aligned with the project stakeholders' view of the system. Feature teams

were formed for key software capabilities and included both development and testing resources. The output of a feature team was a feature description that included a feature overview, any assumptions made by the feature team, prototype UI bitmaps and draft printed reports, a list of relevant scenarios, and draft software requirements.

5. Results

5.1. Lower cost and shorter duration

The two projects, ARCHITECT i2000 (pre-agile) and m2000 (post-agile) are not equal in size. m2000 is a smaller project. Nonetheless, we feel that a comparison of the two projects can demonstrate the effectiveness of the agile approach.

m2000 took 24 months to complete with the initial product offering launching in just 17 months. Following completion of a pilot project, ARCHITECT i2000 took approximately 54 months to launch the initial product offering. We estimate that, if ARCHITECT i2000 were repeated using the agile approach, the overall project duration could be reduced by 20% to 30%.

m2000 had an average staff size of 20 people with peak staffing at 24 software FTEs.. ARCHITECT i2000 peak staffing approached 100 FTEs and probably had an average of around 60 people from inception to initial product launch. We estimate that, if ARCHITECT i2000 were repeated using the agile approach, a headcount reduction of 20% to 30% could have been realized.

In summary, we estimate that using an agile approach on ARCHITECT i2000 would have resulted in a 20% to 30% reduction in both project duration and average team size. In other words, the project would have been implemented more quickly while utilizing a smaller team. The net results would have been a total cost savings of between 35% to 50%.

5.2. Better, less prescriptive test cases

Initially, the m2000 test team applied a similar model for test case definition as was previously done for ARCHITECT i2000. Test cases were written with detailed step-by-step instructions to facilitate execution by naïve users. Although this approach had the distinct

advantage of being able to scale test execution resources, there were unintended less than desirable consequences:

- Test cases were highly coupled to UI navigation. Changes in UI organization or navigation required analogous changes to test cases, even though the intent of the test case and the function of the associated software were not impacted.
- Documentation of test case execution was onerous. In many instances, independent auditors disapproved executed test cases, when specific UI navigation steps were not recorded as complete, even though the intent of the case was satisfied and the software functioned as intended.
- Highly prescriptive test cases resulted in verification of a single execution pathway. The software was essentially hardened to a limited subset of scenarios.

To address the deficiencies, the m2000 test team refined the test case creation practice. The updated practice focused the test case author on the intent of the test case. The detailed set of actions required to navigate the UI and complete a specific software function were replaced by a single bullet point or step in the test case. Although the resulting process required test cases to be executed by subject matter experts, the benefits were considerable:

- Test case development was more efficient. Authors were focused on the intent of the test case, not meticulously describing step-by-step process of running the application. The less prescriptive test cases were easier to understand and debug.
- First pass acceptance of executed test cases by the auditing function increased reducing rework or reruns.
- Alternate execution paths were much more likely to be exercised each time a test case was executed increasing code coverage.
- Test cases were in many cases immune to UI centric application changes.

5.3. Benefits of iteration

5.3.1. Frequent Integration

One of the more significant issues for ARCHITECT i2000 was associated with infrequent integration of embedded and UI / data management applications. The initial integration point occurred months after development was started. This “Big Bang” integration event required several weeks to

complete during which time, the majority of other development activities ground to a halt.

By contrast, the m2000 team generated working software frequently. The continuous build and integration process eliminated the “Big Bang” integration experienced on the ARCHITECT i2000 project.

Additionally, frequent integration between software and hardware elements provided early identification of issues to the hardware team.

5.3.2. Scope management

The iterative approach allowed the m2000 team to effectively manage product scope and limit feature creep. Features or scenarios were frequently evaluated from a stakeholder perspective (Must have, Nice to have, Other). Additionally, the team assessed complexity and related risk. Development focus was on “Must have” features or risk reduction.

At completion of an iteration, the m2000 team would quantify the amount of work completed and assess organizational velocity. The velocity metric (scenarios per iteration) allowed the team to effectively estimate project duration and negotiate trade-offs with key stakeholders.

At time of commercial launch, a number of features, once thought by some stakeholders to be essential, were not included. Some were deferred for as long as three years. Nonetheless, the product was considered highly successful and trading off “Nice to have” features for three years of sales is an easy choice.

5.3.3. Changing requirements

Development of a complex medical device typically takes three to five years. During that development cycle, it is highly likely that requirements will change or additional requirements will emerge. The m2000 project employed a strategy where detailed software requirements were developed and approved as the associated features were designed and implemented. The nearly concurrent execution of feature centric development activities significantly reduced requirements churn in comparison to ARCHITECT i2000 and provided the m2000 team the ability to address new requirements.

5.3.4. Quality

As one would expect from an agile project, testing was performed throughout the m2000 project by developers and test engineers. The continuous availability of working software facilitated ongoing testing. By contrast, in a typical waterfall project the verification is an end-loaded process.

At completion of an incremental software release, both ARCHITECT i2000 and m2000 executed a formal testing phase. Fewer defects were found in formal tests at the end of the m2000 project than the ARCHITECT i2000 project, requiring the team to generate fewer candidate release builds. The availability of working software, early on in the project, was a significant contributor for this reduction in defects.

6. Conclusion

Both ARCHITECT i2000 and m2000 have been key product offerings for their respective businesses and considered commercial successes. From a software development perspective, the projects were significantly different in size and employed very different processes and tools. Threads linking the two projects were limited to a common set of resources engaged in delivering a medical device in a highly regulated environment.

For the most part, there are no hard metrics presented, but more or less a qualitative assessment. How does one gauge the validity of such an assessment?

When comparing a Porsche 911 to an SUV, it's not necessary to see the instrument panel to understand that one vehicle outperforms the other, provides superior acceleration and handling, is more agile. You simply need to experience it.

7. References

- [1] *Abbott 2006 Annual Report*, 2006.
- [2] <http://www.agiletek.com/approach/methodology>, 2009.